

Machine Learning Algorithms Laboratory

Experiment 2

Email Spam/Ham Classification using Naive Bayes and KNN

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naive Bayes and KNN
Student Name	Harshita B
Register Number	3122247001021
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16th August 2026

1 Objective

The objective of this experiment is to classify emails into spam and ham categories using different machine learning algorithms. Naive Bayes and K-Nearest Neighbour (KNN) classifiers are implemented and evaluated using standard classification metrics.

The experiment also aims to:

- Compare Gaussian, Multinomial and Bernoulli Naive Bayes.
- Study the effect of different values of k in KNN.
- Compare KNN performance for different neighbourhood sizes.
- Perform hyperparameter tuning using GridSearchCV and RandomizedSearchCV.
- Compare KDTree and BallTree for nearest-neighbour search.
- Perform five-fold cross-validation.
- Compare training and prediction time.

2 Problem Statement

The task is to classify email messages into two classes: spam and ham. The dataset contains numerical features extracted from email messages.

A classification model is trained using the available labelled data and then used to predict the class of previously unseen emails.

The performance of the models is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

3 Dataset Description

The experiment uses the Spambase dataset.

Dataset Property	Description
Dataset Name	Spambase
Number of Samples	4601
Number of Features	57
Number of Classes	2
Target Variable	class
Class 0	Ham
Class 1	Spam
Train-Test Split	80:20 stratified split
Missing Values	Checked during preprocessing

4 Exploratory Data Analysis

Exploratory Data Analysis was performed before model training to understand the structure and characteristics of the dataset.

The following aspects were examined:

- Dataset dimensions
- Statistical summary
- Missing values
- Class distribution
- Correlation between features
- Feature distributions

4.1 Class Distribution

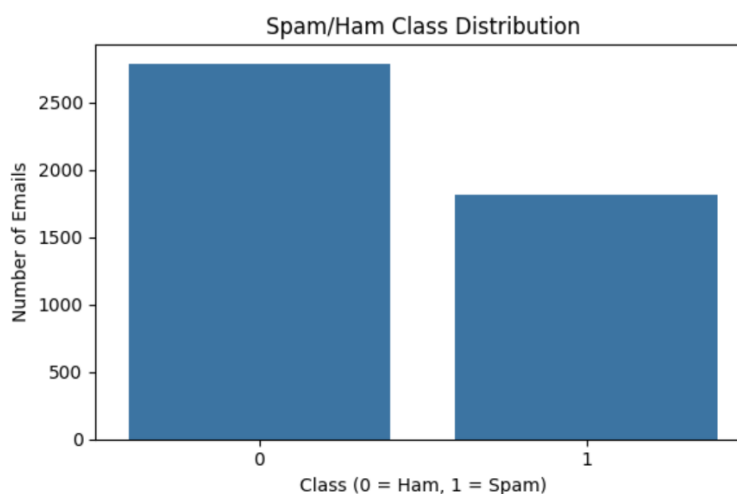


Figure 1: Distribution of spam and ham emails

Observation:

The class distribution graph shows the number of spam and ham emails present in the dataset. The two classes are not perfectly balanced, so accuracy alone may not completely describe the classification performance. Therefore, precision, recall, F1-score and ROC-AUC are also considered.

4.2 Correlation Matrix

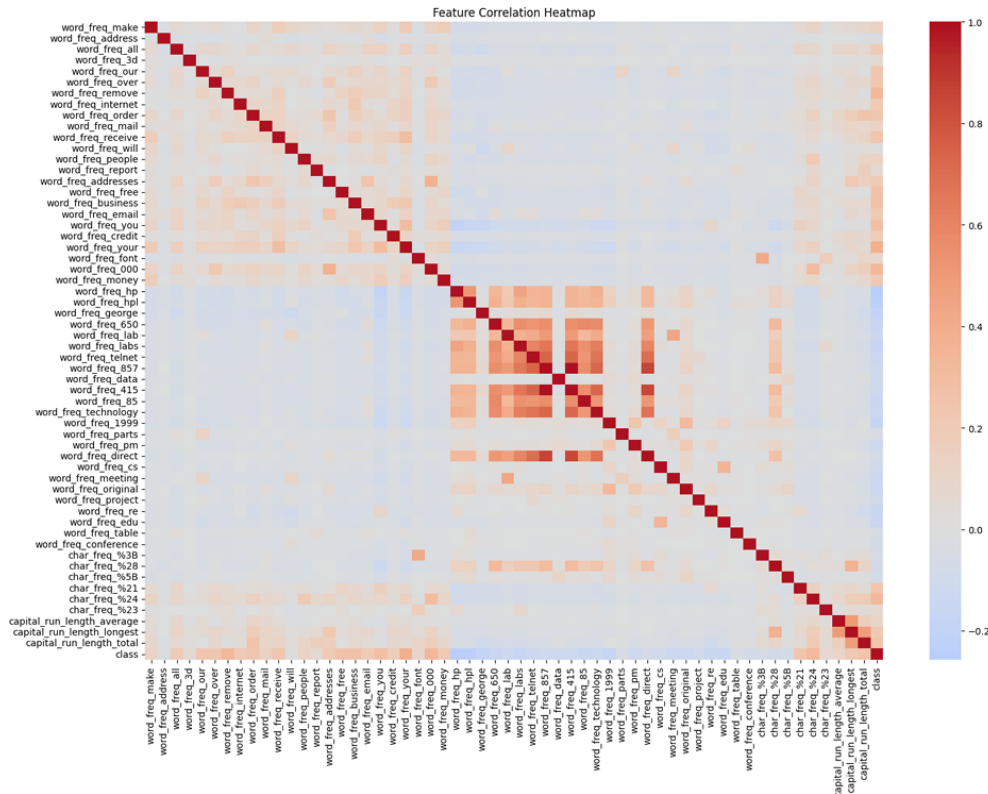


Figure 2: Correlation matrix of dataset features

Observation:

The correlation matrix represents the relationship between different numerical features. The diagonal elements have a correlation value of one because each feature is perfectly correlated with itself. Some features show stronger relationships with each other, while many feature pairs have relatively weak correlation.

4.3 Feature Distribution

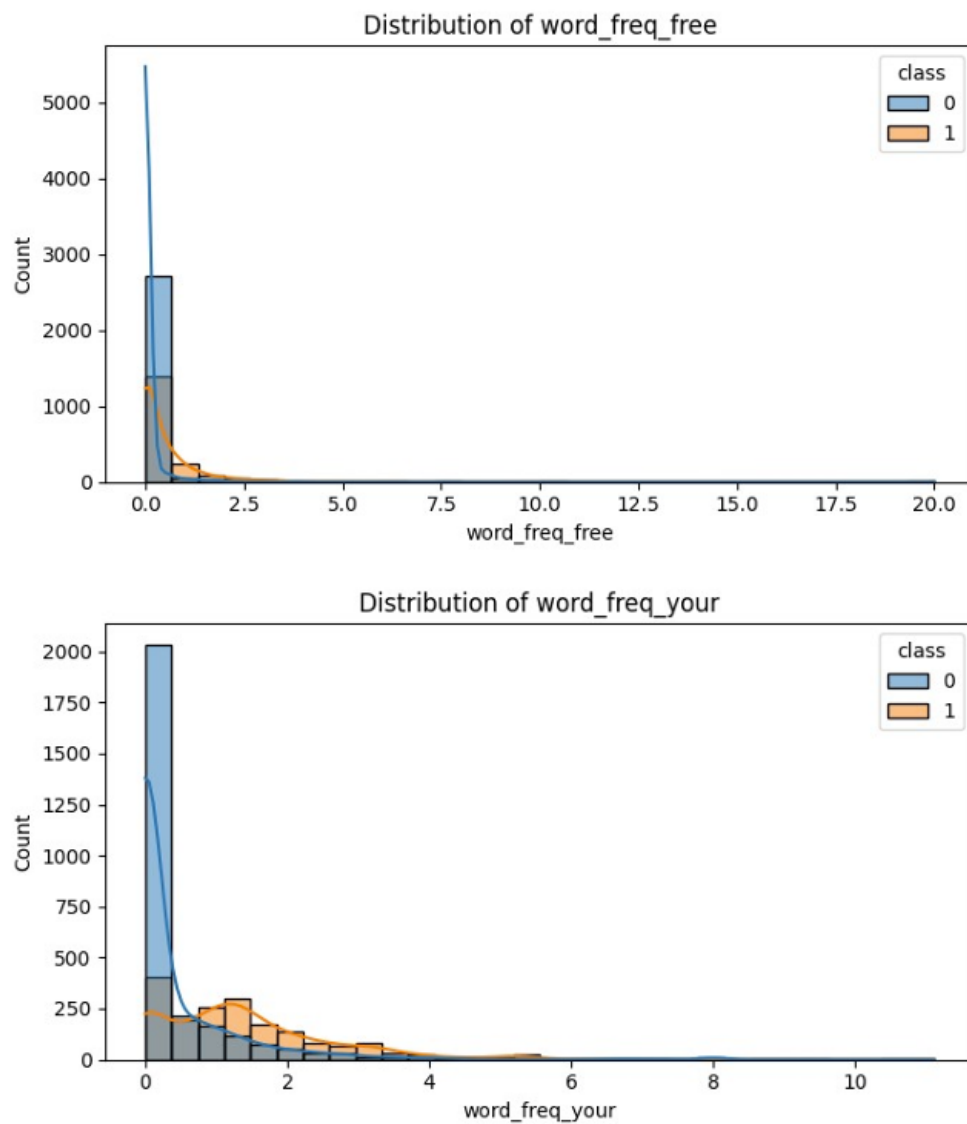


Figure 3: Distribution of selected features

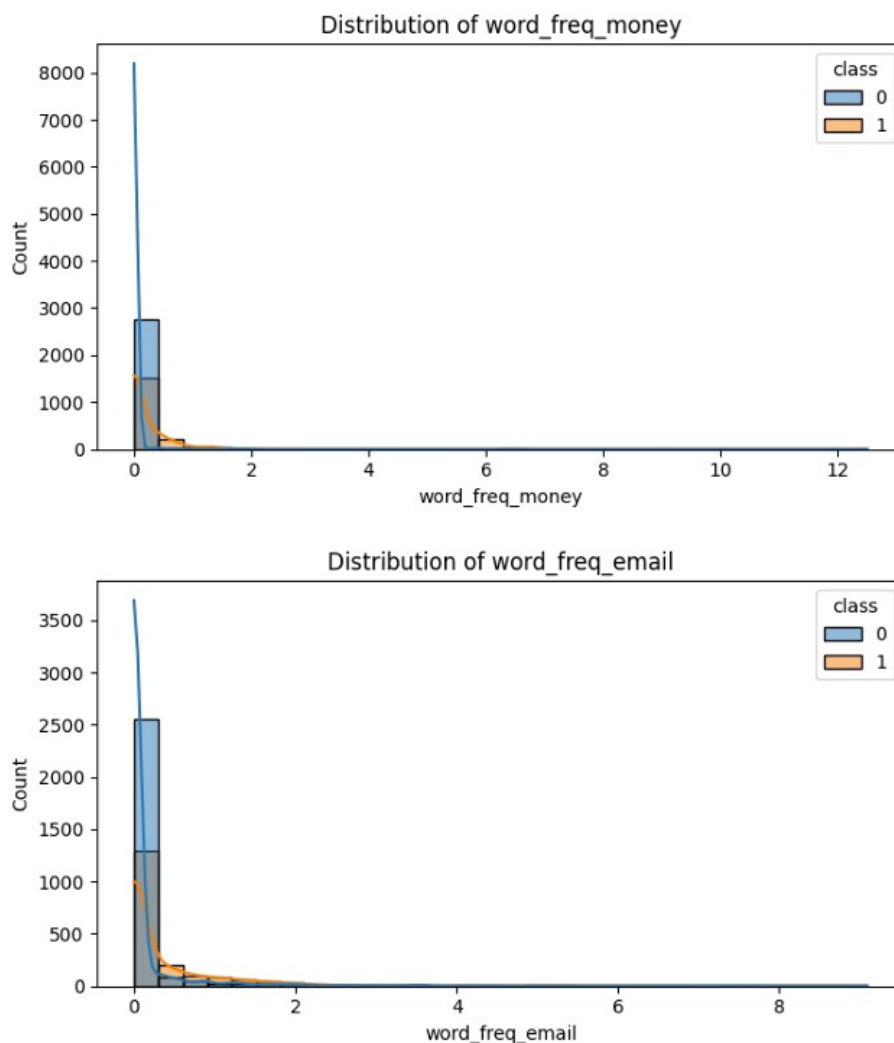


Figure 4: Distribution of selected features

Observation:

The feature distribution plots show how the values of selected features are distributed across the dataset. Some features are concentrated near zero, while a smaller number of observations contain relatively larger values. Such differences in distributions can provide useful information for distinguishing spam from ham emails.

5 Data Preprocessing

The following preprocessing operations were performed:

1. The dataset was loaded using Pandas.
2. The dataset was checked for missing values.
3. The target variable was separated from the input features.
4. The dataset was divided into training and testing sets.

5. Stratification was used to maintain the class distribution.
6. Feature scaling was applied where required.

For KNN, feature scaling is important because the algorithm uses distances between samples.

Standardization was performed using:

$$z = \frac{x - \mu}{\sigma}$$

where x is the original feature value, μ is the mean and σ is the standard deviation.

6 Machine Learning Algorithms

6.1 Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem.

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where:

- C represents the class.
- X represents the input feature vector.
- $P(C|X)$ is the posterior probability.
- $P(X|C)$ is the likelihood.
- $P(C)$ is the prior probability.

The three variants considered in this experiment are:

- Gaussian Naive Bayes
- Multinomial Naive Bayes
- Bernoulli Naive Bayes

6.2 K-Nearest Neighbour

KNN classifies a new sample according to the classes of its nearest training samples.

The Minkowski distance is given by:

$$d(x, y) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$$

For $p = 1$, the distance corresponds to Manhattan distance.

For $p = 2$, the distance corresponds to Euclidean distance.

Different values of k were tested:

$$k \in \{1, 3, 5, 7, 9, 11\}$$

7 Experimental Setup

Python Version	As used in the experiment
Scikit-Learn	As used in the experiment
IDE	Jupyter Notebook
Dataset	Spambase
Train-Test Split	80:20
Cross-Validation	5-fold

8 Results

8.1 Naive Bayes Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8328	0.7146	0.9587	0.8188	0.9376
Multinomial NB	0.8958	0.9349	0.7906	0.8567	0.9612
Bernoulli NB	0.8795	0.8684	0.8182	0.8426	0.9457

Observation:

Among the three Naive Bayes models, Multinomial Naive Bayes achieved the highest accuracy of approximately 89.58%. It also obtained the highest precision and ROC-AUC among the three Naive Bayes models. Gaussian Naive Bayes obtained the highest recall, meaning that it detected a larger proportion of spam samples, although its precision was lower.

8.2 KNN Performance for Different Values of k

k	Accuracy	Precision	Recall	F1-score	ROC-AUC
1	0.8990	0.8792	0.8623	0.8707	0.8926
3	0.8979	0.8726	0.8678	0.8702	0.9360
5	0.9077	0.8861	0.8788	0.8824	0.9506
7	0.9088	0.8930	0.8733	0.8830	0.9566
9	0.9088	0.8930	0.8733	0.8830	0.9602
11	0.9099	0.9000	0.8678	0.8836	0.9622

Observation:

The KNN model achieved its highest accuracy for $k = 11$, with an accuracy of approximately 90.99%. The F1-score was also highest at $k = 11$. The ROC-AUC increased considerably as the value of k increased, reaching 0.9622 for $k = 11$. Therefore, $k = 11$ was selected as the best value among the tested configurations.

8.3 Accuracy versus k

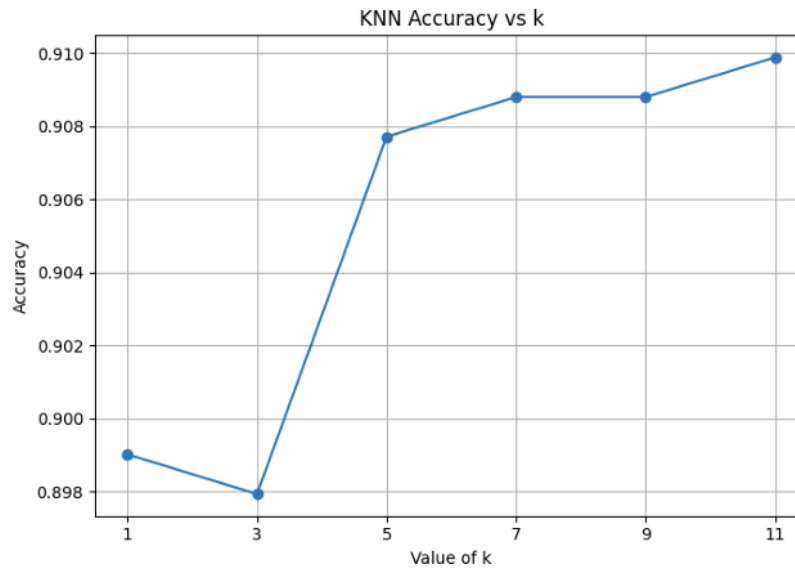


Figure 5: KNN accuracy for different values of k

Inference:

The accuracy generally improves as k increases from 1 to 11. The highest accuracy is obtained at $k = 11$. The small changes between $k = 7$, $k = 9$ and $k = 11$ indicate that the model performance becomes relatively stable for larger neighbourhood sizes.

8.4 Overall Classifier Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8328	0.7146	0.9587	0.8188	0.9376
Multinomial NB	0.8958	0.9349	0.7906	0.8567	0.9612
Bernoulli NB	0.8795	0.8684	0.8182	0.8426	0.9457
KNN ($k = 11$)	0.9099	0.9000	0.8678	0.8836	0.9622

Observation:

KNN with $k = 11$ achieved the highest accuracy among the compared models, with a value of approximately 90.99%. It also achieved the highest F1-score and ROC-AUC. Multinomial Naive Bayes obtained the highest precision, while Gaussian Naive Bayes achieved the highest recall.

8.5 KDTree versus BallTree

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
KDTree	0.9099	0.03863	0.32900
BallTree	0.9099	0.02679	0.31920

Observation:

Both KDTree and BallTree produced the same accuracy of approximately 90.99%. BallTree required less training time and prediction time than KDTree in this experiment. The measured training time was approximately 0.0268 seconds for BallTree compared with 0.0386 seconds for KDTree. BallTree also had a slightly lower prediction time.

Therefore, BallTree was faster for the given dataset while maintaining the same classification accuracy.

8.6 Training and Prediction Time

Model	Training Time (s)	Prediction Time (s)
Gaussian NB	0.03763	0.02083
Multinomial NB	0.01725	0.01288
Bernoulli NB	0.02453	0.01223
KNN ($k = 11$)	0.01034	0.22069

Observation:

KNN required the least training time among the models listed, taking approximately 0.0103 seconds. However, its prediction time was significantly higher at approximately 0.2207 seconds. This is expected because KNN performs distance calculations during prediction rather than learning a complex parametric model during training.

Multinomial Naive Bayes also showed relatively low training and prediction times. Bernoulli Naive Bayes had the lowest prediction time among the Naive Bayes variants.

8.7 Confusion Matrix

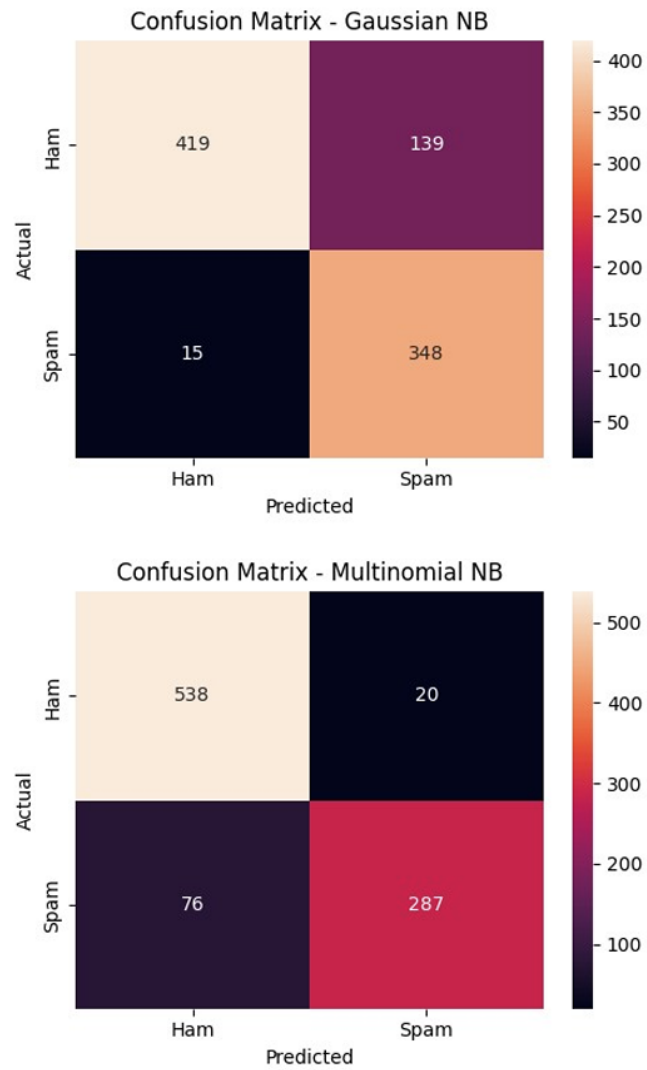


Figure 6: Confusion matrix of the selected classifier

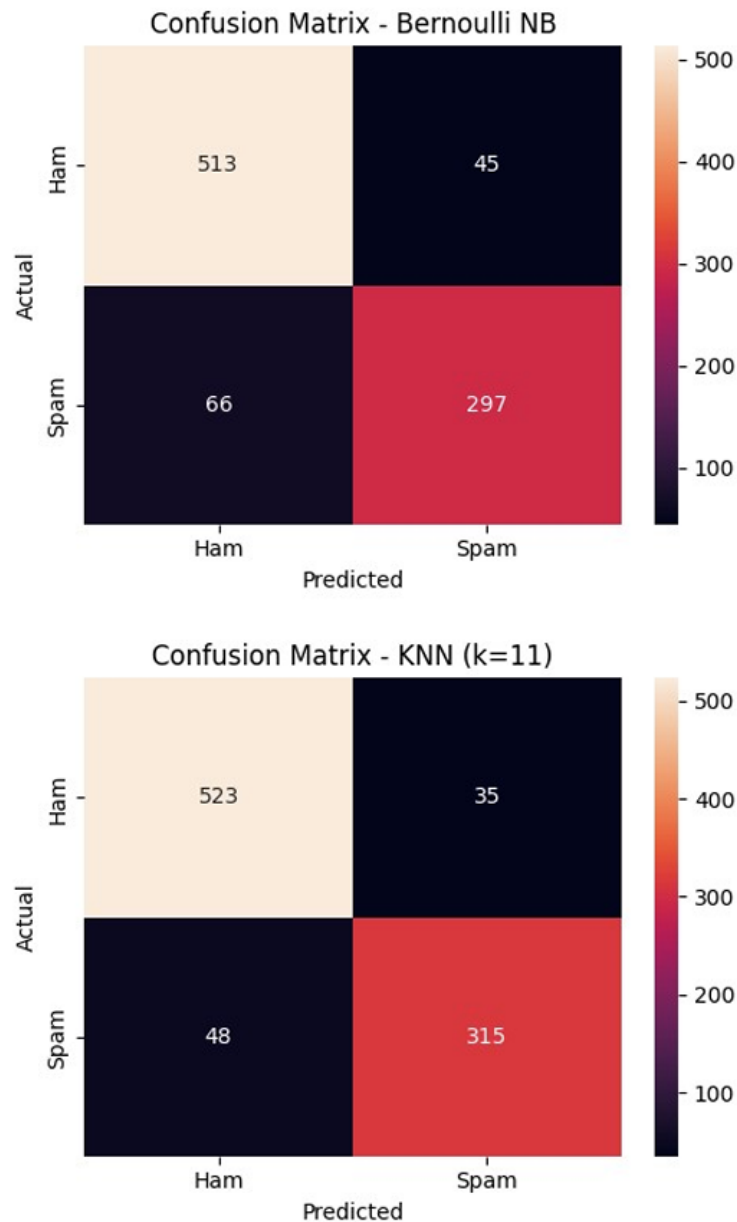


Figure 7: Confusion matrix of the selected classifier

Inference:

The confusion matrix shows the number of correctly and incorrectly classified emails. The diagonal elements represent correct predictions, while the off-diagonal elements represent classification errors. The matrix provides a clearer view of false positives and false negatives than accuracy alone.

8.8 ROC Curve

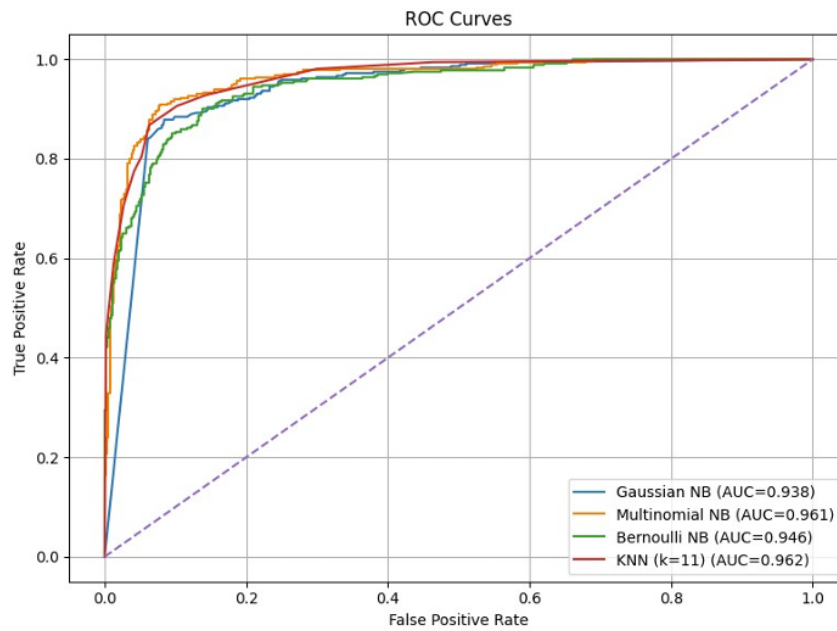


Figure 8: ROC curves of the evaluated classifiers

Inference:

The ROC curve represents the relationship between the true positive rate and false positive rate for different classification thresholds. A curve closer to the upper-left corner indicates better classification performance. KNN with $k = 11$ achieved the highest ROC-AUC among the compared models with a value of approximately 0.9622.

8.9 Precision-Recall Curve

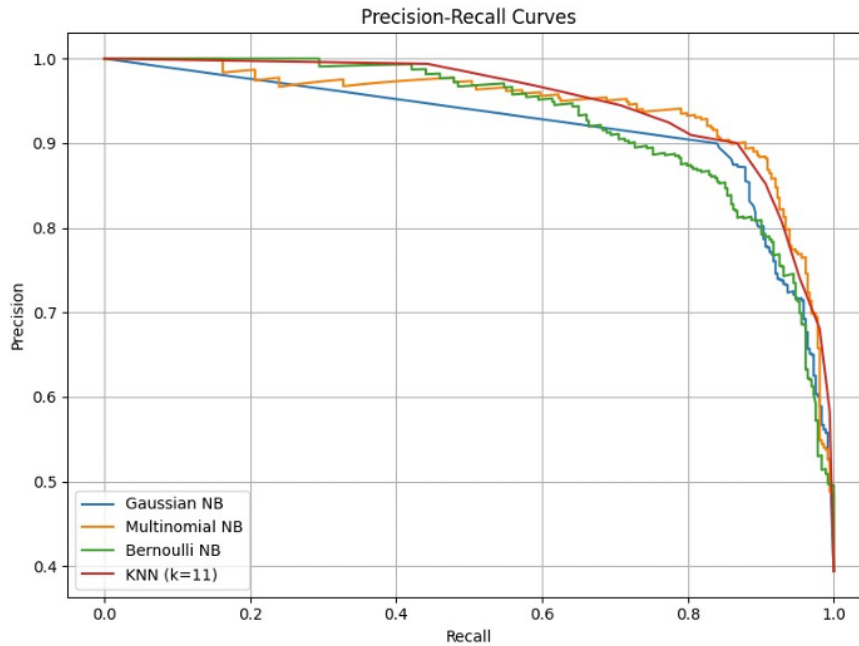


Figure 9: Precision-Recall curves of the evaluated classifiers

Inference:

The Precision-Recall curve shows the trade-off between precision and recall at different classification thresholds. Higher precision means that fewer predicted spam emails are actually ham, while higher recall means that more spam emails are detected. This curve is useful for understanding the types of classification errors made by the model.

9 Hyperparameter Selection

For KNN, different values of k were evaluated to study the effect of the neighbourhood size on classification performance. The tested values were:

$$k = \{1, 3, 5, 7, 9, 11\}$$

Additional KNN parameters included the distance metric, weighting strategy, and neighbour-search algorithm.

GridSearchCV and RandomizedSearchCV were used to systematically search for suitable hyperparameter combinations using five-fold cross-validation.

9.1 GridSearchCV

GridSearchCV was used to evaluate different combinations of KNN hyperparameters using five-fold cross-validation. The parameter search included the number of neighbours, weighting method, distance parameter, and neighbour-search algorithm.

The parameter grid contained 72 possible combinations:

$$6 \times 2 \times 3 \times 2 = 72$$

Since five-fold cross-validation was used, the total number of model fits was:

$$72 \times 5 = 360$$

The best configuration obtained from GridSearchCV was $k = 7$ with distance-based weighting, Manhattan distance ($p = 1$), and the automatic algorithm selection.

Parameter	Value
Best k	7
Weights	distance
Distance parameter p	1
Algorithm	auto
Best CV Accuracy	0.900025
Execution Time	134.8249 seconds

Observation:

GridSearchCV evaluated all specified combinations of KNN hyperparameters using five-fold cross-validation. The best configuration was obtained with $k = 7$, distance weighting, Manhattan distance ($p = 1$), and the automatic algorithm. The best cross-validation accuracy was approximately 90.00%. The complete grid search required approximately 134.82 seconds to evaluate 360 model fits.

9.2 RandomizedSearchCV

RandomizedSearchCV was used to evaluate a selected number of randomly chosen parameter combinations instead of evaluating every possible combination.

In this experiment, 20 parameter configurations were evaluated using five-fold cross-validation. Therefore, the total number of model fits was:

$$20 \times 5 = 100$$

The best configuration obtained from RandomizedSearchCV was $k = 18$, with distance-based weighting, Euclidean distance ($p = 2$), and the KDTree algorithm.

Parameter	Value
Best k	18
Weights	distance
Distance parameter p	2
Algorithm	kd.tree
Best CV Accuracy	0.900457
Number of Iterations	20
Execution Time	26.1578 seconds

Observation:

RandomizedSearchCV obtained a best configuration with $k = 18$, distance-based weighting, Euclidean distance ($p = 2$), and KDTree. The best cross-validation accuracy was approximately 90.05%. The execution time was approximately 26.16 seconds.

Compared with GridSearchCV, RandomizedSearchCV required considerably less time because it evaluated only 20 randomly selected configurations instead of evaluating all 72 configurations.

9.3 GridSearchCV and RandomizedSearchCV Comparison

Method	Best CV Accuracy	Execution Time (s)	Best k
GridSearchCV	0.900025	134.8249	7
RandomizedSearchCV	0.900457	26.1578	18

Observation:

RandomizedSearchCV achieved a slightly higher best cross-validation accuracy than GridSearchCV in this experiment. It also required much less execution time. GridSearchCV took approximately 134.82 seconds, whereas RandomizedSearchCV took approximately 26.16 seconds.

The difference in execution time is mainly due to the number of configurations evaluated. GridSearchCV evaluated 360 fits, whereas RandomizedSearchCV evaluated 100 fits.

10 Five-Fold Cross-Validation

Five-fold cross-validation was used to evaluate the consistency of the classification models. The data was divided into five folds. In each iteration, four folds were used for training and the remaining fold was used for validation. This process was repeated until every fold had been used as the validation set.

10.1 Naive Bayes Cross-Validation

The fold accuracies obtained for the Naive Bayes model were:

Fold	Accuracy
1	0.820847
2	0.808696
3	0.801087
4	0.821739
5	0.825000
Average	0.815474

Observation:

The Naive Bayes model achieved an average five-fold cross-validation accuracy of approximately 81.55%. The individual fold accuracies ranged from approximately 80.11% to 82.50%. The relatively small variation between the folds indicates that the model showed reasonably consistent performance across the different validation subsets.

10.2 KNN Cross-Validation

The fold accuracies obtained for the KNN model were:

Fold	Accuracy
1	0.902280
2	0.906522

3	0.920652
4	0.908696
5	0.892391
Average	0.906108

Observation:

The KNN model achieved an average five-fold cross-validation accuracy of approximately 90.61%. The highest accuracy was obtained in Fold 3, with an accuracy of approximately 92.07%, while the lowest accuracy was obtained in Fold 5, with approximately 89.24%. The relatively small difference between the fold accuracies indicates reasonably consistent performance across the validation subsets.

10.3 Cross-Validation Comparison

Model	Average CV Accuracy
Naive Bayes	0.815474
KNN	0.906108

Observation:

The KNN model achieved a higher average five-fold cross-validation accuracy than the Naive Bayes model. KNN obtained an average accuracy of approximately 90.61%, whereas Naive Bayes obtained approximately 81.55%. Based on the cross-validation results, KNN showed better validation performance on this dataset.

10.4 KDTree versus BallTree

The nearest-neighbour search algorithms KDTree and BallTree were also compared using the KNN classifier.

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
KDTree	0.909881	0.038629	0.329003
BallTree	0.909881	0.026792	0.319200

Observation:

Both KDTree and BallTree achieved the same classification accuracy of approximately 90.99%. However, BallTree required less training time and prediction time in this experiment. The training time of BallTree was approximately 0.0268 seconds compared with 0.0386 seconds for KDTree. Similarly, BallTree required approximately 0.3192 seconds for prediction compared with 0.3290 seconds for KDTree.

Therefore, BallTree was slightly faster than KDTree for the given dataset while maintaining the same classification accuracy.

11 Complexity Analysis

Let n represent the number of training samples and d represent the number of features.

Algorithm	Training	Prediction
Naive Bayes	Approximately $O(nd)$	Low computational cost
KNN	Very low training cost	Distance calculation for neighbours
KDTree	Approximately $O(n \log n)$	Efficient neighbour search under suitable conditions
BallTree	Approximately $O(n \log n)$	Efficient neighbour search under suitable conditions

The theoretical complexity gives an indication of how the algorithms scale with the size of the dataset. Actual execution time also depends on the implementation, hardware and characteristics of the data.

12 Discussion

The experiment compared Naive Bayes and KNN for spam and ham email classification.

Among the Naive Bayes models, Multinomial Naive Bayes performed best in terms of accuracy, precision and ROC-AUC. It achieved an accuracy of approximately 89.58%.

KNN showed an improvement in performance as the value of k increased in the tested range. The best result was obtained for $k = 11$, with an accuracy of approximately 90.99%.

KNN with $k = 11$ also obtained the highest F1-score and ROC-AUC among the classifiers compared in this experiment. However, its prediction time was considerably higher than the Naive Bayes models.

The KDTree and BallTree implementations achieved identical accuracy. BallTree was faster in both training and prediction time for the given experiment.

These results demonstrate that model selection should not depend only on accuracy. Computational cost and the type of errors produced by the classifier should also be considered.

13 Analysis Questions

13.1 Which Naive Bayes variant performed best?

Multinomial Naive Bayes performed best among the Naive Bayes variants in terms of accuracy. It achieved an accuracy of 0.8958 and an ROC-AUC of 0.9612.

13.2 What is the optimal value of k ?

Among the tested values, $k = 11$ produced the highest KNN accuracy of 0.9099. It also produced the highest F1-score of approximately 0.8836.

13.3 Which model achieved the highest accuracy?

KNN with $k = 11$ achieved the highest accuracy among the compared models, with an accuracy of approximately 90.99%.

13.4 Which model had the highest recall?

Gaussian Naive Bayes achieved the highest recall among the compared models, with a recall of approximately 0.9587.

13.5 Which model had the highest precision?

Multinomial Naive Bayes achieved the highest precision, with a value of approximately 0.9349.

13.6 KDTree versus BallTree

Both algorithms achieved the same accuracy of approximately 90.99%. BallTree was faster, requiring approximately 0.0268 seconds for training and 0.3192 seconds for prediction, compared with 0.0386 seconds and 0.3290 seconds for KDTree.

13.7 GridSearchCV versus RandomizedSearchCV

GridSearchCV evaluates all combinations in the specified parameter grid, while RandomizedSearchCV evaluates a selected number of randomly chosen configurations. The actual execution times and best configurations should be reported from the corresponding outputs.

14 Conclusion

The experiment successfully demonstrated the use of Naive Bayes and KNN algorithms for spam and ham email classification.

Gaussian, Multinomial and Bernoulli Naive Bayes were compared. Multinomial Naive Bayes provided the best accuracy among the Naive Bayes variants, while Gaussian Naive Bayes achieved the highest recall.

Different values of k were evaluated for KNN. The best performance was obtained at $k = 11$, giving an accuracy of approximately 90.99%, F1-score of 0.8836 and ROC-AUC of 0.9622.

KDTree and BallTree produced the same classification accuracy, while BallTree required slightly less training and prediction time.

Overall, KNN with $k = 11$ produced the highest classification accuracy in this experiment, although its prediction time was higher than that of the Naive Bayes models.

15 Observation Record

Experiment Component	Observation
Dataset	Spambase dataset containing 4601 samples and 57 features.
Classification Task	Binary classification of emails into ham and spam.
Naive Bayes	Gaussian, Multinomial and Bernoulli variants were compared.

Best Naive Bayes Model	Multinomial Naive Bayes with accuracy 89.58%.
KNN Values	$k = 1, 3, 5, 7, 9, 11$.
Best KNN	KNN with $k = 11$.
Best KNN Accuracy	90.99%.
Best KNN F1-score	0.8836.
Best KNN ROC-AUC	0.9622.
Best Recall	Gaussian Naive Bayes with recall 0.9587.
Best Precision	Multinomial Naive Bayes with precision 0.9349.
KDTree Accuracy	90.99%.
BallTree Accuracy	90.99%.
Faster Tree Method	BallTree for the measured dataset.
KNN Training Time	0.01034 seconds for $k = 11$.
KNN Prediction Time	0.22069 seconds for $k = 11$.

16 References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed., Morgan Kaufmann, 2016.
4. UCI Machine Learning Repository, “Spambase Dataset.”

A Python Code

If the Python file is in the same folder as this LaTeX file, it can be included using:

```

1 # The complete Python code used for Experiment 2
2 # should be placed in experiment2.py

1 # -*- coding: utf-8 -*-
2 """Experiment_2.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1ji3fyxyXPN5MPxaUvvAX0uwBkG85T1uu
8
9 **Email Spam/Ham Classification using Naive Bayes and
10 KNN**
11
12
13 **Objectives:**
14 * Build spam classifiers using Nave Bayes and KNN.
15 * Compare Gaussian, Multinomial and Bernoulli Nave Bayes.
```

```
16 * Study the effect of varying k.
17 * Compare KDTree and BallTree
18 """
19
20 #Importing all the packages
21 import time
22 import numpy as np
23 import pandas as pd
24 import matplotlib.pyplot as plt
25 import seaborn as sns
26
27 from sklearn.model_selection import (
28     train_test_split,
29     cross_val_score,
30     GridSearchCV,
31     RandomizedSearchCV,
32     StratifiedKFold
33 )
34
35 from sklearn.preprocessing import StandardScaler, MinMaxScaler
36 from sklearn.pipeline import Pipeline
37
38 from sklearn.naive_bayes import (
39     GaussianNB,
40     MultinomialNB,
41     BernoulliNB
42 )
43
44 from sklearn.neighbors import KNeighborsClassifier
45
46 from sklearn.metrics import (
47     accuracy_score,
48     precision_score,
49     recall_score,
50     f1_score,
51     roc_auc_score,
52     confusion_matrix,
53     classification_report,
54     roc_curve,
55     precision_recall_curve
56 )
57
58 #Loading the dataset
59 df = pd.read_csv("spambase_csv.csv")
60
61 print("Dataset shape:", df.shape)
62 print("\nFirst five rows:")
63 print(df.head())
64
65 print("\nDataset information:")
66 print(df.info())
67
68 #Check for missing values in the dataset print("\nMissing values:")
69 print(df.isnull().sum().sum())
70
71 # If missing values are present, replace them using
72 # the median value of the corresponding numerical column.
73 for column in df.columns:
```

```
74     if df[column].isnull().sum() > 0:
75         df[column] = df[column].fillna(df[column].median())
76
77 print("\nMissing values after handling:",
78       df.isnull().sum().sum())
79
80 #Separate features and target
81 X = df.drop("class", axis=1)
82 y = df["class"]
83
84 print("\nNumber of features:", X.shape[1])
85 print("Number of samples:", X.shape[0])
86
87 #Exploratory Data Analysis
88 print("\nClass distribution:")
89 print(y.value_counts())
90
91 # Plot class distribution.
92 plt.figure(figsize=(6, 4))
93 sns.countplot(x=y)
94 plt.title("Spam/Ham Class Distribution")
95 plt.xlabel("Class (0 = Ham, 1 = Spam)")
96 plt.ylabel("Number of Emails")
97 plt.tight_layout()
98 plt.show()
99
100 #Correlation Heatmap
101 plt.figure(figsize=(16, 12))
102 sns.heatmap(
103     df.corr(),
104     cmap="coolwarm",
105     center=0
106 )
107 plt.title("Feature Correlation Heatmap")
108 plt.tight_layout()
109 plt.show()
110
111 # 6. Histograms of selected features
112
113 important_features = [
114     "word_freq_free",
115     "word_freq_your",
116     "word_freq_money",
117     "word_freq_email"
118 ]
119
120 for feature in important_features:
121     if feature in df.columns:
122         plt.figure(figsize=(7, 4))
123         sns.histplot(
124             data=df,
125             x=feature,
126             hue="class",
127             kde=True,
128             bins=30
129         )
130         plt.title(f"Distribution of {feature}")
131         plt.tight_layout()
```

```
132     plt.show()
133
134 # 7. Boxplots
135 for feature in important_features:
136     if feature in df.columns:
137         plt.figure(figsize=(7, 4))
138         sns.boxplot(
139             data=df,
140             x="class",
141             y=feature
142         )
143         plt.title(f"Boxplot of {feature}")
144         plt.xlabel("Class (0 = Ham, 1 = Spam)")
145         plt.tight_layout()
146         plt.show()
147
148 # 8. Train-test split
149 # Stratification keeps approximately the same proportion of spam and ham messages in
150 # both sets.
151 X_train, X_test, y_train, y_test = train_test_split(
152     X,
153     y,
154     test_size=0.20,
155     random_state=42,
156     stratify=y
157 )
158 print("\nTraining samples:", X_train.shape[0])
159 print("Testing samples:", X_test.shape[0])
160
161 # 9. Feature scaling
162 # StandardScaler is used for Gaussian NB and KNN.
163 scaler = StandardScaler()
164
165 X_train_scaled = scaler.fit_transform(X_train)
166 X_test_scaled = scaler.transform(X_test)
167
168 # MinMaxScaler is useful for Multinomial and Bernoulli NB
169 # because these models work better with non-negative values.
170 minmax_scaler = MinMaxScaler()
171
172 X_train_minmax = minmax_scaler.fit_transform(X_train)
173 X_test_minmax = minmax_scaler.transform(X_test)
174
175 # 10. Function for calculating evaluation metrics
176 def evaluate_model(model_name, y_true, y_pred, y_prob):
177     """
178     Calculate the main classification metrics and
179     return them as a dictionary.
180     """
181
182     results = {
183         "Model": model_name,
184         "Accuracy": accuracy_score(y_true, y_pred),
185         "Precision": precision_score(y_true, y_pred),
186         "Recall": recall_score(y_true, y_pred),
187         "F1": f1_score(y_true, y_pred),
188         "ROC-AUC": roc_auc_score(y_true, y_prob)
```

```
189     }
190
191     return results
192
193 # Naive Bayes Models
194
195 nb_results = []
196 nb_models = {}
197
198 # -----
199 # Gaussian Naive Bayes
200 # -----
201
202 start_time = time.perf_counter()
203
204 gaussian_nb = GaussianNB()
205 gaussian_nb.fit(X_train_scaled, y_train)
206
207 gaussian_train_time = time.perf_counter() - start_time
208
209 start_time = time.perf_counter()
210
211 gaussian_pred = gaussian_nb.predict(X_test_scaled)
212 gaussian_prob = gaussian_nb.predict_proba(X_test_scaled)[: , 1]
213
214 gaussian_prediction_time = time.perf_counter() - start_time
215
216 nb_results.append(
217     evaluate_model(
218         "Gaussian NB",
219         y_test,
220         gaussian_pred,
221         gaussian_prob
222     )
223 )
224
225 nb_models["Gaussian NB"] = gaussian_nb
226
227
228 # -----
229 # Multinomial Naive Bayes
230 # -----
231
232 start_time = time.perf_counter()
233
234 multinomial_nb = MultinomialNB()
235 multinomial_nb.fit(X_train_minmax, y_train)
236
237 multinomial_train_time = time.perf_counter() - start_time
238
239 start_time = time.perf_counter()
240
241 multinomial_pred = multinomial_nb.predict(X_test_minmax)
242 multinomial_prob = multinomial_nb.predict_proba(X_test_minmax)[: , 1]
243
244 multinomial_prediction_time = time.perf_counter() - start_time
245
246 nb_results.append(
```

```

247     evaluate_model(
248         "Multinomial NB",
249         y_test,
250         multinomial_pred,
251         multinomial_prob
252     )
253 )
254
255 nb_models["Multinomial NB"] = multinomial_nb
256
257
258 # -----
259 # Bernoulli Naive Bayes
260 # -----
261
262 start_time = time.perf_counter()
263
264 bernoulli_nb = BernoulliNB()
265 bernoulli_nb.fit(X_train_minmax, y_train)
266
267 bernoulli_train_time = time.perf_counter() - start_time
268
269 start_time = time.perf_counter()
270
271 bernoulli_pred = bernoulli_nb.predict(X_test_minmax)
272 bernoulli_prob = bernoulli_nb.predict_proba(X_test_minmax)[: , 1]
273
274 bernoulli_prediction_time = time.perf_counter() - start_time
275
276 nb_results.append(
277     evaluate_model(
278         "Bernoulli NB",
279         y_test,
280         bernoulli_pred,
281         bernoulli_prob
282     )
283 )
284
285 nb_models["Bernoulli NB"] = bernoulli_nb
286
287
288 # Convert results into a DataFrame.
289 nb_results_df = pd.DataFrame(nb_results)
290
291 print("\n===== Naive Bayes Comparison =====")
292 print(nb_results_df.to_string(index=False))
293
294 # KNN for different values of k
295 knn_results = []
296
297 k_values = [1, 3, 5, 7, 9, 11]
298
299 for k in k_values:
300
301     # Standardized features are used because KNN is
302     # distance-based.
303     knn = KNeighborsClassifier(
304         n_neighbors=k,

```

```

305     weights="uniform",
306     algorithm="auto"
307 )
308
309 start_time = time.perf_counter()
310
311 knn.fit(X_train_scaled, y_train)
312
313 train_time = time.perf_counter() - start_time
314
315 start_time = time.perf_counter()
316
317 pred = knn.predict(X_test_scaled)
318 prob = knn.predict_proba(X_test_scaled)[: , 1]
319
320 prediction_time = time.perf_counter() - start_time
321
322 knn_results.append({
323     "k": k,
324     "Accuracy": accuracy_score(y_test, pred),
325     "Precision": precision_score(y_test, pred),
326     "Recall": recall_score(y_test, pred),
327     "F1": f1_score(y_test, pred),
328     "ROC-AUC": roc_auc_score(y_test, prob),
329     "Training Time": train_time,
330     "Prediction Time": prediction_time
331 })
332
333 knn_results_df = pd.DataFrame(knn_results)
334
335 print("\n===== KNN Comparison =====")
336 print(knn_results_df.to_string(index=False))
337
338 # -----
339 # 13. Accuracy vs k
340 # -----
341
342 plt.figure(figsize=(7, 5))
343
344 plt.plot(
345     knn_results_df["k"],
346     knn_results_df["Accuracy"],
347     marker="o"
348 )
349
350 plt.xlabel("Value of k")
351 plt.ylabel("Accuracy")
352 plt.title("KNN Accuracy vs k")
353 plt.xticks(k_values)
354 plt.grid(True)
355 plt.tight_layout()
356 plt.show()
357
358
359 # Find the best k based on accuracy.
360 best_k = knn_results_df.loc[
361     knn_results_df["Accuracy"].idxmax(),

```

```
363     "k"
364 ]
365
366 print("\nBest k:", best_k)
367
368 # Best KNN model
369
370 best_knn = KNeighborsClassifier(
371     n_neighbors=int(best_k),
372     weights="uniform",
373     algorithm="auto"
374 )
375
376 start_time = time.perf_counter()
377 best_knn.fit(X_train_scaled, y_train)
378 best_knn_train_time = time.perf_counter() - start_time
379
380 start_time = time.perf_counter()
381
382 best_knn_pred = best_knn.predict(X_test_scaled)
383 best_knn_proba = best_knn.predict_proba(X_test_scaled)[: , 1]
384
385 best_knn_prediction_time = time.perf_counter() - start_time
386
387 print("\n===== Best KNN =====")
388
389 print(
390     classification_report(
391         y_test,
392         best_knn_pred,
393         target_names=["Ham", "Spam"]
394     )
395 )
396
397 # Confusion matrices
398 model_predictions = {
399     "Gaussian NB": gaussian_pred,
400     "Multinomial NB": multinomial_pred,
401     "Bernoulli NB": bernoulli_pred,
402     f"KNN (k={best_k})": best_knn_pred
403 }
404
405 for name, predictions in model_predictions.items():
406
407     cm = confusion_matrix(y_test, predictions)
408
409     plt.figure(figsize=(5, 4))
410
411     sns.heatmap(
412         cm,
413         annot=True,
414         fmt="d",
415         xticklabels=["Ham", "Spam"],
416         yticklabels=["Ham", "Spam"]
417     )
418
419     plt.title(f"Confusion Matrix - {name}")
420     plt.xlabel("Predicted")
```

```
421     plt.ylabel("Actual")
422     plt.tight_layout()
423     plt.show()
424
425     # =====
426     # 16. ROC Curves
427     # =====
428
429     model_probabilities = {
430         "Gaussian NB": gaussian_prob,
431         "Multinomial NB": multinomial_prob,
432         "Bernoulli NB": bernoulli_prob,
433         f"KNN (k={best_k})": best_knn_prob
434     }
435
436     plt.figure(figsize=(8, 6))
437
438     for name, probabilities in model_probabilities.items():
439
440         fpr, tpr, _ = roc_curve(
441             y_test,
442             probabilities
443         )
444
445         auc_value = roc_auc_score(
446             y_test,
447             probabilities
448         )
449
450         plt.plot(
451             fpr,
452             tpr,
453             label=f"{name} (AUC={auc_value:.3f})"
454         )
455
456     plt.plot(
457         [0, 1],
458         [0, 1],
459         linestyle="--"
460     )
461
462     plt.xlabel("False Positive Rate")
463     plt.ylabel("True Positive Rate")
464     plt.title("ROC Curves")
465     plt.legend()
466     plt.grid(True)
467     plt.tight_layout()
468     plt.show()
469
470     #Precision-Recall Curves
471
472     plt.figure(figsize=(8, 6))
473
474     for name, probabilities in model_probabilities.items():
475
476         precision, recall, _ = precision_recall_curve(
477             y_test,
478             probabilities
```

```
479     )
480
481     plt.plot(
482         recall,
483         precision,
484         label=name
485     )
486
487 plt.xlabel("Recall")
488 plt.ylabel("Precision")
489 plt.title("Precision-Recall Curves")
490 plt.legend()
491 plt.grid(True)
492 plt.tight_layout()
493 plt.show()
494
495 #GridSearchCV for KNN
496
497 # A pipeline ensures that scaling is performed separately
498 # inside each cross-validation fold.
499 knn_pipeline = Pipeline([
500     ("scaler", StandardScaler()),
501     ("knn", KNeighborsClassifier())
502 ])
503
504 parameter_grid = {
505     "knn__n_neighbors": [1, 3, 5, 7, 9, 11],
506     "knn__weights": ["uniform", "distance"],
507     "knn__algorithm": ["auto", "kd_tree", "ball_tree"],
508     "knn__p": [1, 2]
509 }
510
511 start_time = time.perf_counter()
512
513 grid_search = GridSearchCV(
514     estimator=knn_pipeline,
515     param_grid=parameter_grid,
516     cv=5,
517     scoring="accuracy",
518     n_jobs=-1
519 )
520
521 grid_search.fit(X, y)
522
523 grid_time = time.perf_counter() - start_time
524
525 print("\n===== GridSearchCV =====")
526 print("Best parameters:", grid_search.best_params_)
527 print("Best CV accuracy:", grid_search.best_score_)
528 print("Execution time:", grid_time)
529
530 #RandomizedSearchCV
531
532 from scipy.stats import randint
533
534 random_parameter_grid = {
535     "knn__n_neighbors": randint(1, 20),
536     "knn__weights": ["uniform", "distance"],
```

```
537     "knn__algorithm": ["auto", "kd_tree", "ball_tree"],
538     "knn__p": [1, 2]
539 }
540
541 start_time = time.perf_counter()
542
543 random_search = RandomizedSearchCV(
544     estimator=knn_pipeline,
545     param_distributions=random_parameter_grid,
546     n_iter=20,
547     cv=5,
548     scoring="accuracy",
549     random_state=42,
550     n_jobs=-1
551 )
552
553 random_search.fit(X, y)
554
555 random_time = time.perf_counter() - start_time
556
557 print("\n===== RandomizedSearchCV =====")
558 print("Best parameters:", random_search.best_params_)
559 print("Best CV accuracy:", random_search.best_score_)
560 print("Execution time:", random_time)
561
562 #KDTree vs BallTree
563 tree_results = []
564
565 for algorithm in ["kd_tree", "ball_tree"]:
566
567     tree_knn = Pipeline([
568         ("scaler", StandardScaler()),
569         (
570             "knn",
571             KNeighborsClassifier(
572                 n_neighbors=int(best_k),
573                 algorithm=algorithm
574             )
575         )
576     ])
577
578     start_time = time.perf_counter()
579
580     tree_knn.fit(X_train, y_train)
581
582     training_time = time.perf_counter() - start_time
583
584     start_time = time.perf_counter()
585
586     tree_pred = tree_knn.predict(X_test)
587
588     prediction_time = time.perf_counter() - start_time
589
590     tree_accuracy = accuracy_score(
591         y_test,
592         tree_pred
593     )
594
```

```

595     tree_results.append({
596         "Algorithm": algorithm,
597         "Accuracy": tree_accuracy,
598         "Training Time": training_time,
599         "Prediction Time": prediction_time
600     })
601
602
603 tree_results_df = pd.DataFrame(tree_results)
604
605 print("\n===== KDTree vs BallTree =====")
606 print(tree_results_df.to_string(index=False))
607
608 # =====
609 # 21. Five-fold Cross Validation
610 # =====
611
612 cv = StratifiedKFold(
613     n_splits=5,
614     shuffle=True,
615     random_state=42
616 )
617
618 cv_results = []
619
620 # Best Naive Bayes model based on test accuracy.
621 best_nb_name = nb_results_df.loc[
622     nb_results_df["Accuracy"].idxmax(),
623     "Model"
624 ]
625
626 best_nb_model = nb_models[best_nb_name]
627
628 # For simplicity, use the already scaled data for NB CV.
629 # Gaussian NB is evaluated on standardized features.
630 nb_cv_scores = cross_val_score(
631     GaussianNB(),
632     scaler.fit_transform(X),
633     y,
634     cv=cv,
635     scoring="accuracy"
636 )
637
638 # Evaluate the selected KNN for each fold.
639 knn_cv_pipeline = Pipeline([
640     ("scaler", StandardScaler()),
641     (
642         "knn",
643         KNeighborsClassifier(
644             n_neighbors=int(best_k)
645         )
646     )
647 ])
648
649 knn_cv_scores = cross_val_score(
650     knn_cv_pipeline,
651     X,
652     y,

```

```

653     cv=cv,
654     scoring="accuracy"
655 )
656
657 print("\n===== Five Fold Cross Validation =====")
658
659 print("Naive Bayes fold accuracies:")
660 print(nb_cv_scores)
661
662 print("Naive Bayes average accuracy:",
663       nb_cv_scores.mean())
664
665 print("\nKNN fold accuracies:")
666 print(knn_cv_scores)
667
668 print("KNN average accuracy:",
669       knn_cv_scores.mean())
670
671
672 # Plot CV accuracy.
673 plt.figure(figsize=(8, 5))
674
675 fold_numbers = [1, 2, 3, 4, 5]
676
677 plt.plot(
678     fold_numbers,
679     nb_cv_scores,
680     marker="o",
681     label="Gaussian NB"
682 )
683
684 plt.plot(
685     fold_numbers,
686     knn_cv_scores,
687     marker="o",
688     label=f"KNN (k={best_k})"
689 )
690
691 plt.xlabel("Fold")
692 plt.ylabel("Accuracy")
693 plt.title("Five-Fold Cross Validation Accuracy")
694 plt.xticks(fold_numbers)
695 plt.legend()
696 plt.grid(True)
697 plt.tight_layout()
698 plt.show()
699
700 # =====
701 # 22. Training and prediction time comparison
702 # =====
703
704 time_results = pd.DataFrame({
705     "Algorithm": [
706         "Gaussian NB",
707         "Multinomial NB",
708         "Bernoulli NB",
709         f"KNN (k={best_k})"
710     ],

```

```

711     "Training Time": [
712         gaussian_train_time,
713         multinomial_train_time,
714         bernoulli_train_time,
715         best_knn_train_time
716     ],
717     "Prediction Time": [
718         gaussian_prediction_time,
719         multinomial_prediction_time,
720         bernoulli_prediction_time,
721         best_knn_prediction_time
722     ]
723 })
724
725 print("\n===== Experimental Time Analysis =====")
726 print(time_results.to_string(index=False))
727
728 # Training time plot
729 plt.figure(figsize=(8, 5))
730
731 plt.bar(
732     time_results["Algorithm"],
733     time_results["Training Time"]
734 )
735
736 plt.xlabel("Algorithm")
737 plt.ylabel("Training Time (seconds)")
738 plt.title("Training Time Comparison")
739 plt.xticks(rotation=20)
740 plt.tight_layout()
741 plt.show()
742
743 # Prediction time plot
744 plt.figure(figsize=(8, 5))
745
746 plt.bar(
747     time_results["Algorithm"],
748     time_results["Prediction Time"]
749 )
750
751 plt.xlabel("Algorithm")
752 plt.ylabel("Prediction Time (seconds)")
753 plt.title("Prediction Time Comparison")
754 plt.xticks(rotation=20)
755 plt.tight_layout()
756 plt.show()
757
758 # =====
759 # 23. Overall classifier comparison
760 # =====
761
762 comparison_df = nb_results_df[
763     ["Model", "Accuracy", "Precision", "Recall", "F1", "ROC-AUC"]
764 ].copy()
765
766 knn_best_row = pd.DataFrame([

```

```

769     "Model": f"KNN (k={best_k})",
770     "Accuracy": accuracy_score(y_test, best_knn_pred),
771     "Precision": precision_score(y_test, best_knn_pred),
772     "Recall": recall_score(y_test, best_knn_pred),
773     "F1": f1_score(y_test, best_knn_pred),
774     "ROC-AUC": roc_auc_score(y_test, best_knn_prob)
775 })
776
777 comparison_df = pd.concat(
778     [comparison_df, knn_best_row],
779     ignore_index=True
780 )
781
782 print("\n===== Overall Classifier Comparison =====")
783 print(comparison_df.to_string(index=False))
784
785
786 # Plot overall accuracy comparison.
787 plt.figure(figsize=(8, 5))
788
789 plt.bar(
790     comparison_df["Model"],
791     comparison_df["Accuracy"]
792 )
793
794 plt.xlabel("Classifier")
795 plt.ylabel("Accuracy")
796 plt.title("Classifier Accuracy Comparison")
797 plt.xticks(rotation=20)
798 plt.ylim(0, 1)
799 plt.tight_layout()
800 plt.show()
801
802 # =====
803 # 24. Save important results
804 # =====
805
806 nb_results_df.to_csv(
807     "naive_bayes_results.csv",
808     index=False
809 )
810
811 knn_results_df.to_csv(
812     "knn_results.csv",
813     index=False
814 )
815
816 tree_results_df.to_csv(
817     "tree_results.csv",
818     index=False
819 )
820
821 time_results.to_csv(
822     "time_results.csv",
823     index=False
824 )
825
826 comparison_df.to_csv(

```

```
827     "classifier_comparison.csv",  
828     index=False  
829 )
```